RAMAIAH
Institute of Technology

Unit I – Chapter 3

- Introduction
- Characteristics of Database approach
- Actors on the Scene
- Workers behind the scene
- Advantages of using DBMS approach
- Data models, schemas and instances
- Three-schema architecture and data independence
- Database languages and interfaces
- The database system environment
- Entity-Relationship Model
- Using high level Conceptual data models for Database Design
- A Sample Database Application
- Entity types, Entity sets Attributes and Keys
- Relationship types, Relationship Sets, Roles and Structural Constraints
- Weak Entity Types

Entity-Relationship Model

- **Database application** refers to a particular database and the associated programs that implement the database queries and updates. Ex:- Bank Database
- Part of the database application will require the design, implementation, and testing of **application programs.**
- **Entity-Relationship** (ER) **model,** is a popular high level conceptual data model.
- This model and its variations are frequently used for the conceptual design of database applications, and many database design tools employ its concepts.

USING HIGH-LEVEL CONCEPTUAL DATA MODELS FOR DATABASE DESIGN (1)

Figure 7.1 shows a simplified description of the database design process. The first step shown is requirements collection and analysis. The database designers interview prospective database users to understand and document their data requirements. The result of this step is a concisely written set of users' requirements. These requirements should be specified in as detailed and complete a form as possible. It is useful to specify the known functional requirements of the application.

The next step is to create a conceptual schema for the database, using a high-level conceptual data model. This step is called conceptual design. The conceptual schema is a concise description of **the data requirements of the users** and includes detailed descriptions of the **entity types, relationships, and constraints**; these are expressed using the concepts provided by the high-level data model.

The last step is the physical design phase, during which the **internal storage structures, indexes, access paths, and file organizations** for the database files are specified. In parallel with these activities, application programs are designed and implemented as database transactions corresponding to the high-level transaction specifications.

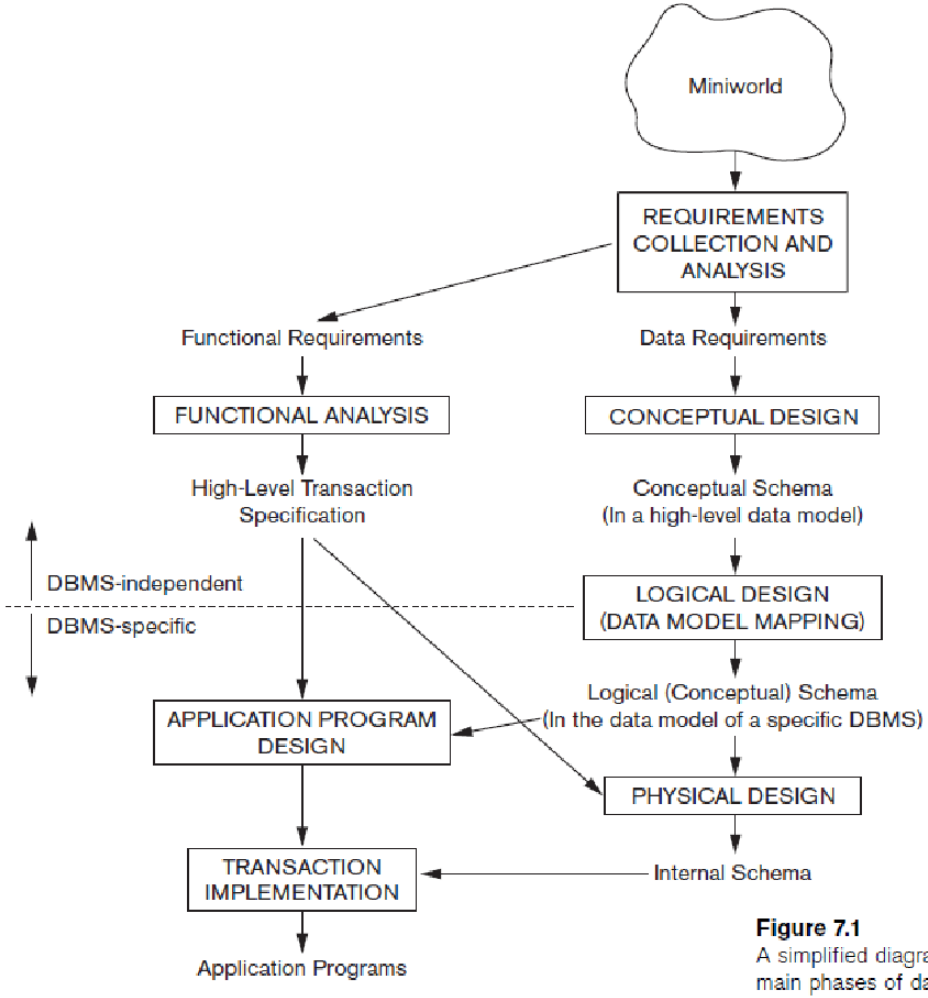


Figure 7.1
A simplified diagram to illustrate the main phases of database design.

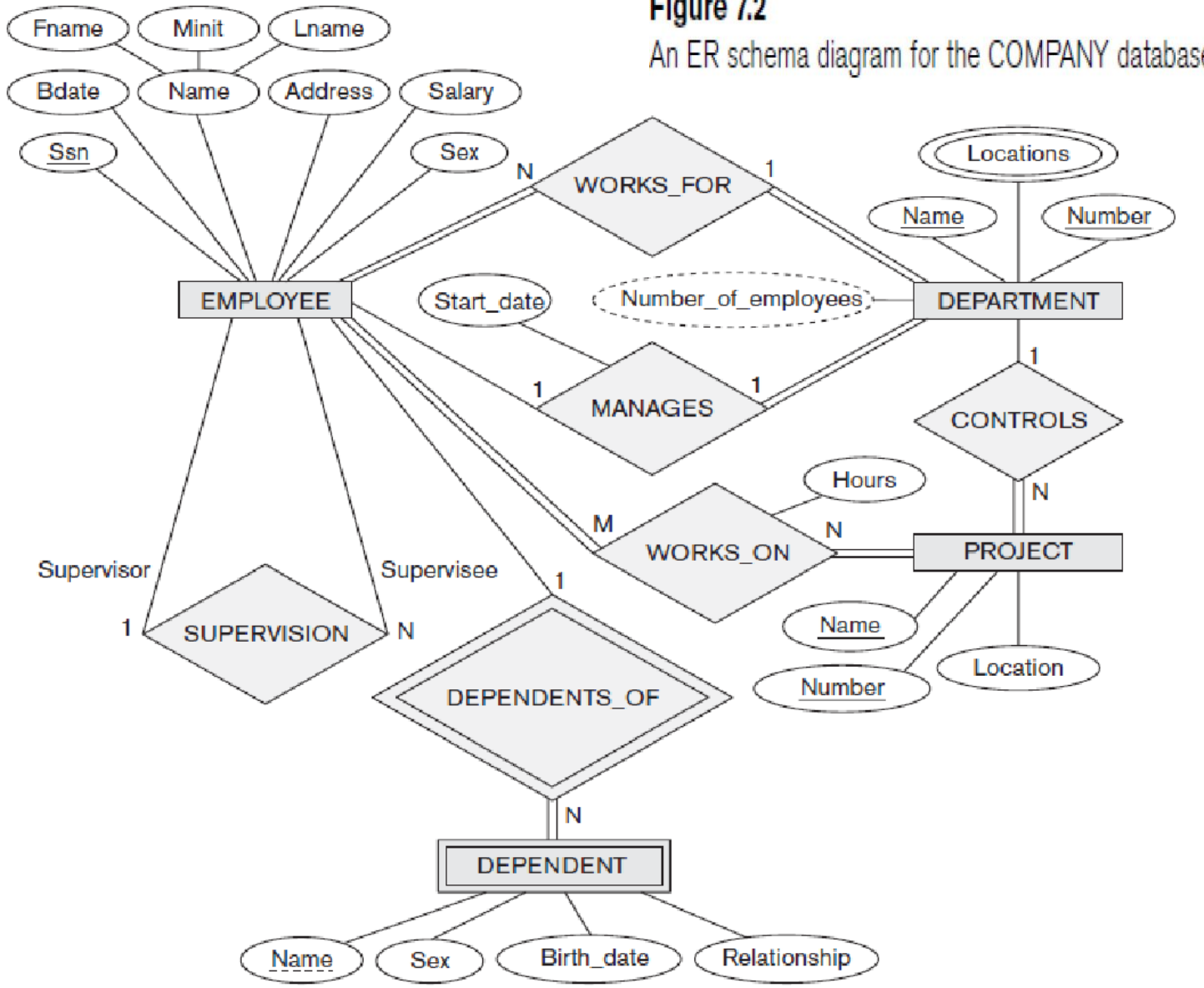
A SAMPLE DATABASE APPLICATION (1)

- An example database application, called COMPANY, that serves to illustrate the basic ER model concepts and their use in schema design.
- The COMPANY database keeps track of a company's employees, departments, and projects.
- Suppose that after the requirements collection and analysis phase, the database designers provided the following description of the "miniworld"-the part of the company to be represented in the database:

A SAMPLE DATABASE APPLICATION (2)

- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- We store each employee's name, social security number, address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date, and relationship to the employee.
- Figure 7.2 shows how the schema for this database application can be displayed by means of the graphical notation known as ER diagrams.

Figure 7.2
An ER schema diagram for the COMPANY database.



ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (1)

Entities and Attributes (1)

- **Entities and Their Attributes.**
- The basic object that the ER model represents is an entity, which is a **"thing" in the real world with an independent existence.**
- An entity may be an object with a physical existence (for example, a particular person, car, house, or employee) or it may be an object with a conceptual existence (for example, a company, a job, or a university course). Each entity has attributes-the particular properties that describe it.
- For example, an employee entity may be described by the employee's name, age, address, salary, and job. A particular entity will have a value for each of its attributes.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (2)

Entities and Attributes (2)

- The attribute values **that describe each entity** become a major part of the data stored in the database.
- Figure 7.3 shows two entities and the values of their attributes.
- The employee entity e1 has four attributes: Name, Address, Age, and HomePhone; their values are "John Smith," "2311 Kirby, Houston, Texas 77001," "55," and "713-749-2630," respectively.
- The company entity c1 has three attributes: Name, Headquarters, and President; their values are "Sunco Oil," "Houston," and "John Smith," respectively.

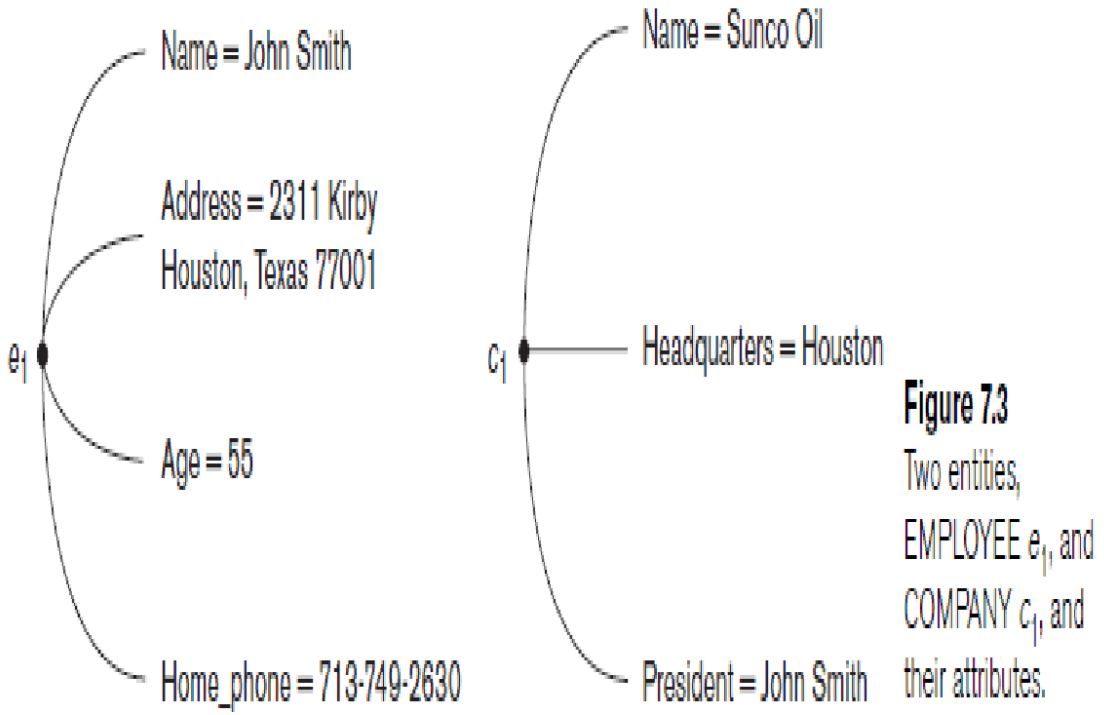


Figure 7.3
Two entities, EMPLOYEE e_1, and COMPANY c_1, and their attributes.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (3)

Entities and Attributes (3)

- Several types of attributes occur in the ER model: simple versus composite, single-valued versus Multivalued, and stored versus derived.
- **Composite versus Simple (Atomic) Attributes**. Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings.
- For example, the Address attribute of the employee entity shown in Figure 7.3 can be subdivided into StreetAddress, City, State, and Zip, with the values "2311 Kirby,"
- "Houston," "Texas," and "77001." Attributes that are not divisible are called simple or atomic attributes.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (4)

Entities and Attributes (4)

- Composite attributes can form a hierarchy; for example, Street Address can be further subdivided into three simple attributes: Number, Street, and Apartment Number, as shown in Figure 3.4.
- The value of a composite attribute is the concatenation of the values of its constituent simple attributes.
- If the composite attribute is referenced only as a whole, there is no need to subdivide it into component attributes.
- For example, if there is no need to refer to the individual components of an address (zip code, street, and so on), then the whole address can be designated as a simple attribute.

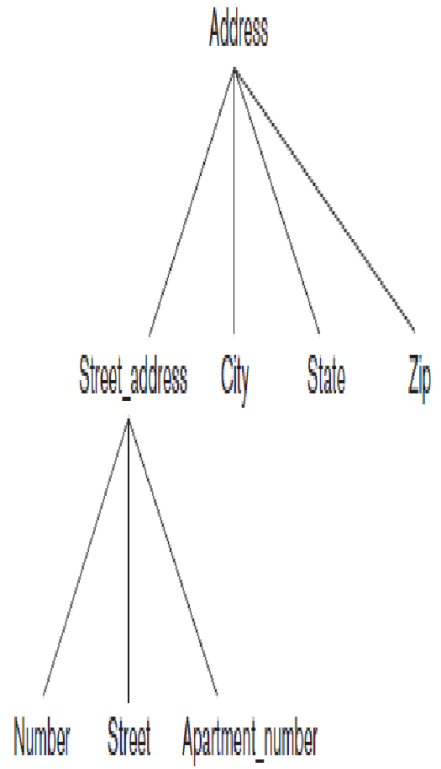


Figure 7.4
A hierarchy of composite attributes.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (5)

Entities and Attributes (5)

Single-Valued versus Multivalued Attributes.

- Most attributes have a single value for a particular entity; such attributes are called single-valued.
- For example, Age is a single-valued attribute of a person.
- In some cases an attribute can have a set of values for the same entity-for example, a Colors attribute for a car, or a College Degrees attribute for a person. Such attributes are called multivalued.
- A multivalued attribute may have lower and upper bounds to constrain the number of values allowed for each individual entity.
- For example, the Colors attribute of a car may have between one and three values, if we assume that a car can have at most three colors.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (6)

Entities and Attributes (6)

Stored versus Derived Attributes.

- In some cases, two (or more) attribute values are related-for example, the Age and BirthDate attributes of a person.
- For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's BirthDate.
- The Age attribute is hence called a derived attribute and is said to be derivable from the BirthDate attribute, which is called a stored attribute.
- Some attribute values can be derived from related entities; for example, an attribute NumberOfEmployees of a department entity can be derived by counting the number of employees related to (working for) that department.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (7)

Entities and Attributes (7)

- Null Values. In some cases a particular entity may not have an applicable value for an attribute.
- For example, the ApartmentNumber attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes.
- Similarly, a CollegeDegrees attribute applies only to persons with college degrees.
- For such situations, a special value called null is created.
- An address of a single-family home would have null for its ApartmentNumber attribute, and a person with no college degree would have null for CollegeDegrees.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (8)

Entities and Attributes (8)

- Null can also be used if we do not know the value of an attribute for a particular entity-for example, if we do not know the blood group of "John Smith".
- The meaning of the former type of null is not applicable, whereas the meaning of the latter is unknown. The "unknown" category of null can be further classified into two cases.
- The first case arises when it is known that the attribute value exists but is missing-for example, if the Height attribute of a person is listed as null.
- The second case arises when it is not known whether the attribute value exists-for example, if the Homephone attribute of a person is null.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (9)

Entities and Attributes (9)

Complex Attributes.

- Notice that composite and multivalued attributes can be nested in an arbitrary way.
- We can represent arbitrary nesting by grouping components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }.
- Such attributes are called complex attributes. For example, if a person can have more than one residence and each residence can have multiple phones, an attribute AddressPhone for a person can be specified as shown in Figure 7.5

{Address_phone({Phone(Area_code,Phone_number)},Address(Street_address (Number,Street,Apartment_number),City,State,Zip))}

Figure 7.5
A complex attribute: Address_phone.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (10)
Entity Types, Entity Sets, Keys, and Value Sets(1)

- **Entity Types and Entity Sets.**

- An **entity type** defines a collection (or set) of entities that have the same attributes.

- Each entity type in the database is described by its name and attributes. Figure 7.6 shows two entity types, named EMPLOYEE and COMPANY, and a list of attributes for each.

- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set**; the entity set is usually referred to using the **same name** as the entity type.

- For example, EMPLOYEE refers to both a type of entity as well as the current set of all employee entities in the database.

- An **entity type** describes the schema or **intension** for a set of entities that share the same structure.

- The collection of entities of a particular entity type are grouped into an **entity set**, which is also called the **extension** of the entity type.

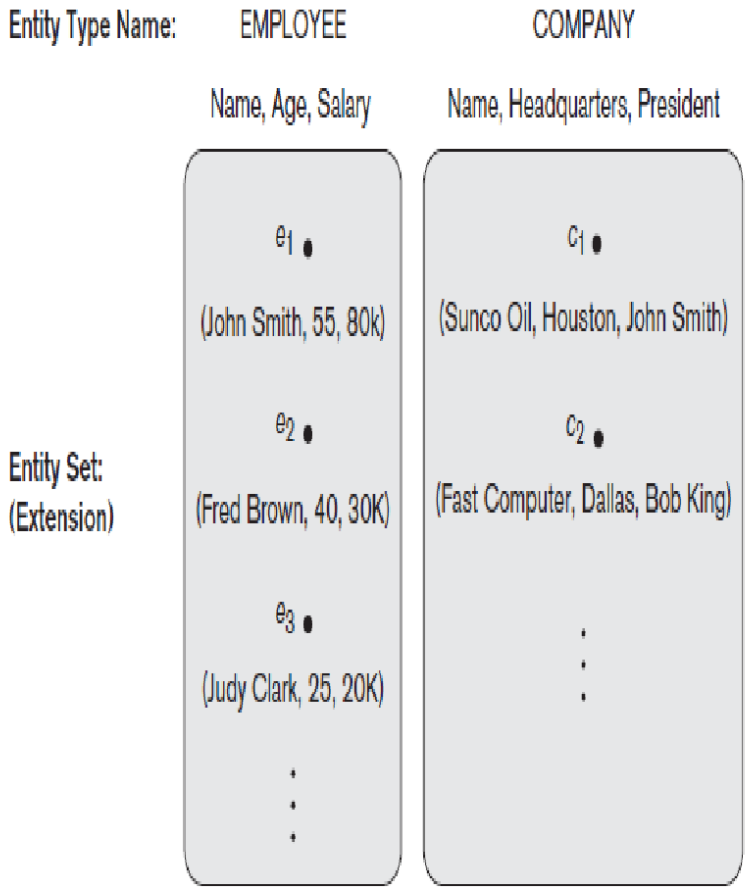


Figure 7.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (12)
Entity Types, Entity Sets, Keys, and Value Sets(2)

Key Attributes of an Entity Type.

- An important constraint on the entities of an entity type is the **key or uniqueness constraint** on attributes.
- An entity type usually has an attribute whose **values are distinct for each individual entity** in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.
- For example, the **Name attribute** is a key of the COMPANY entity type, because no two companies are allowed to have the same name.
- For the **PERSON** entity type, a typical key attribute is **SocialSecurityNumber**.
- Sometimes, several attributes together form a key, meaning that the **combination of the attribute values must be distinct** for each entity.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (13)
Entity Types, Entity Sets, Keys, and Value Sets(3)

- Some entity types have **more than one key attribute**. For example, each of the **VehicleID** and **Registration attributes** of the entity type CAR is a key in its own right.
- The **Registration attribute** is an example of a **composite key** formed from two simple component attributes, RegistrationNumber and State, neither of which is a key on its own.
- An entity type may also have **no key**, in which case it is called a **weak entity type.**

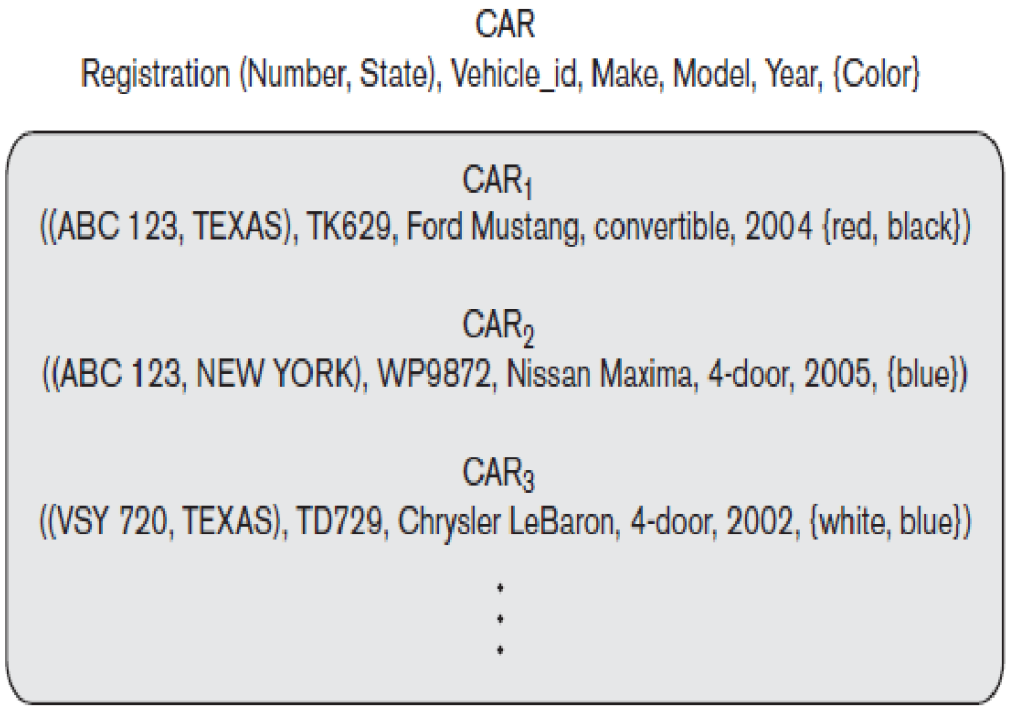


FIGURE 3.7 The CAR entity type with two key attributes, Registration and VehicleID.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (14)
Entity Types, Entity Sets, Keys, and Value Sets(4)

- **Value Sets (Domains) of Attributes.**

- Each simple attribute of an entity type is associated with a **value set (or domain of values),** which specifies the **set of values that may be assigned to that attribute** for each individual entity.

- If the range of **ages** allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of **integer numbers between 16 and 70.**

- Value sets are typically specified using the **basic data types** available in most programming languages, such as **integer, string, boolean, float, enumerated type, subrange, date, time** and so on.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (15)
Entity Types, Entity Sets, Keys, and Value Sets(5)

- We refer to the value of **attribute A for entity e as A(e).**
- The previous definition **covers both single-valued and multivalued attributes**, as well as **nulls**.
- A **null value** is represented by the **empty set.**
- For **single-valued attributes, A(e) is restricted to being a singleton set** for each entity e in E, whereas there is no restriction on multivalued attributes.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (16)
Initial Conceptual Design of the COMPANY Database (1)

- An entity type DEPARTMENT with attributes Name, Number, Locations, Manager, and ManagerStartDate. Locations is the only multivalued attribute. We can specify that both Name and Number are (separate) key attributes, because each was specified to be unique.
- An entity type PROJECT with attributes Name, Number, Location, and ControllingDepartment. Both Name and Number are (separate) key attributes.
- An entity type EMPLOYEE with attributes Name, SSN (for social security number), Sex, Address, Salary, BirthDate, Department, and Supervisor. Both Name and Address may be composite attributes; however, this was not specified in the requirements. We must go back to the users to see if any of them will refer to the individual components of Name-FirstName, Middlelnitial, LastName-or of Address.

ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS (17)
Initial Conceptual Design of the COMPANY Database (2)

- An entity type DEPENDENT with attributes Employee, DependentName, Sex, BirthDate, and Relationship (to the employee).
- So far, we have not represented the fact that an employee can work on several projects, nor have we represented the number of hours per week an employee works on each project.
- It can be represented by a multivalued composite attribute of EMPLOYEE called WorksOn with the simple components (Project, Hours).
- Alternatively, it can be represented as a multivalued composite attribute of PROJECT called Workers with the simple components (Employee, Hours).
- The Name attribute of EMPLOYEE is shown as a composite attribute

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (1)

- Whenever an attribute of one entity type refers to another entity type, some relationship exists. For example, the attribute Manager of DEPARTMENT refers to an employee who manages the department
- The attribute Controlling Department of PROJECT refers to the department that controls the project
- The attribute Supervisor of EMPLOYEE refers to another employee (the one who supervises this employee)
- The attribute Department of EMPLOYEE refers to the department for which the employee works; and so on.
- In the ER model, these references should not be represented as attributes but as relationships.
- In the initial design of entity types, relationships are typically captured in the form of attributes.
- As the design is refined, these attributes get converted into relationships between entity types.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (2)

Relationship Types, Sets, and Instances (1)

- A **relationship** type **R** among n entity types E_1, E_2, ... , En defines a **set of associations or a relationship** set-among entities from these entity types.
- A **relationship type** and its corresponding **relationship set** are customarily **referred to by the same name, R.**

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (3)

Relationship Types, Sets, and Instances (2)

- For example, consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department for which the employee works.
- Each relationship instance in the relationship set WORKSFOR associates one employee entity and one department entity.
- Figure 7.9 illustrates this example

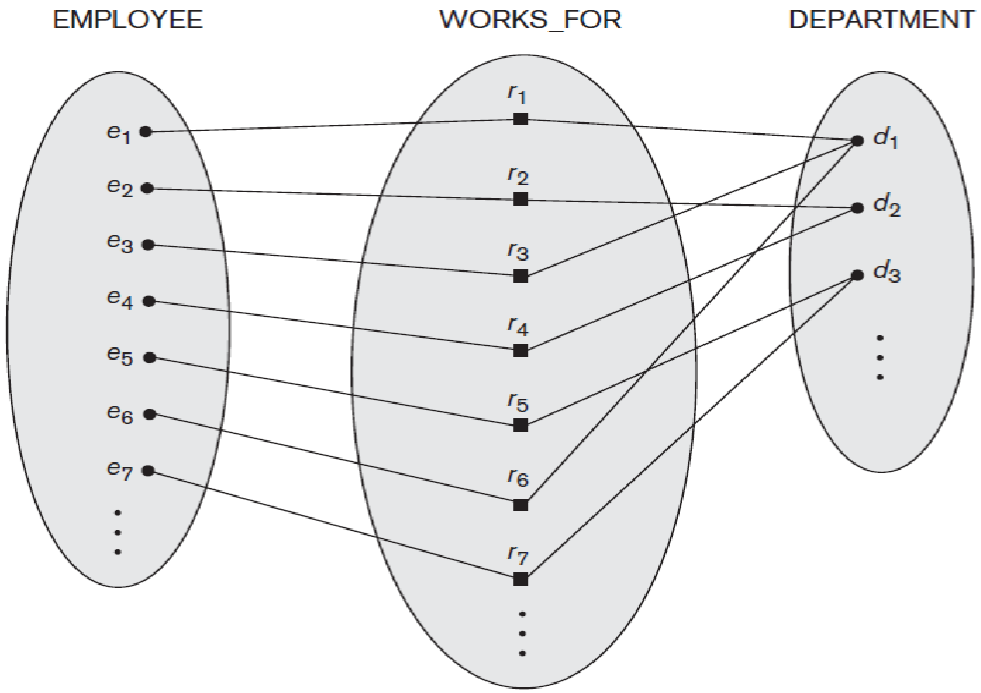


Figure 7.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (5)

Relationship Degree, Role Names, and Recursive Relationships (1)

Degree of a Relationship Type.

- The degree of a relationship type is the **number of participating entity types.**
- Hence, the WORKSFOR relationship is of **degree two**.
- A relationship type of **degree two is called binary**, and one of **degree three is called ternary.**
- An example of a ternary relationship is SUPPLY, where each relationship instance r_i associates three entities-a supplier s, a part p, and a project whenever s supplies part p to project j.
- Relationships can generally be of any degree

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (6)

Relationship Degree, Role Names, and Recursive Relationships (2)

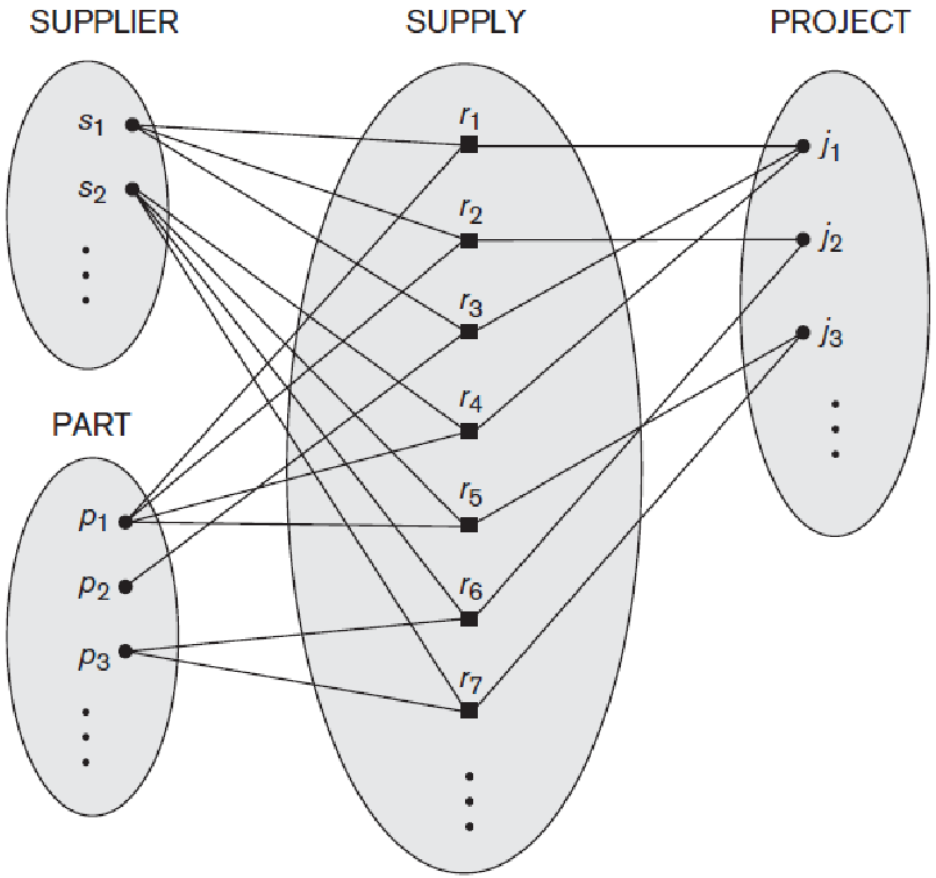


Figure 7.10
Some relationship instances in the SUPPLY ternary relationship set.

Relationship Degree, Role Names, and Recursive Relationships (3)

- **Relationships as Attributes.**

- It is sometimes convenient to think of a relationship type in terms of attributes.
- Consider the WORKS_FOR relationship type. One can think of an attribute called Department of the EMPLOYEE entity type whose value for each employee entity is (a reference to) the department entity that the employee works for.
- Hence, the value set for this Department attribute is the set of all DEPARTMENT entities, which is the DEPARTMENT entity set.
- However, when we think of a binary relationship as an attribute, we always have two options.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (8)

Relationship Degree, Role Names, and Recursive Relationships (4)

Role Names and Recursive Relationships.

- Each entity type that participates in a relationship type plays a particular **role** in the relationship.
- The **role name** signifies the **role** that a participating entity from the entity type plays in each relationship instance.
- For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.
- **Role names are not technically necessary** in relationship types where all the participating entity types are **distinct**, since each participating entity type name can be used as the role name.
- However, in some cases the **same entity type participates more than once in a relationship type** in different roles. In such cases the role name becomes essential for distinguishing the meaning of each participation. Such relationship types are called **recursive relationships**.
- The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity type.
- Hence, the EMPLOYEE entity type participates twice in SUPERVISION: once in the role of **supervisor** (or boss), and once in the role of **supervisee** (or subordinate).

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (9)

Relationship Degree, Role Names, and Recursive Relationships (5)

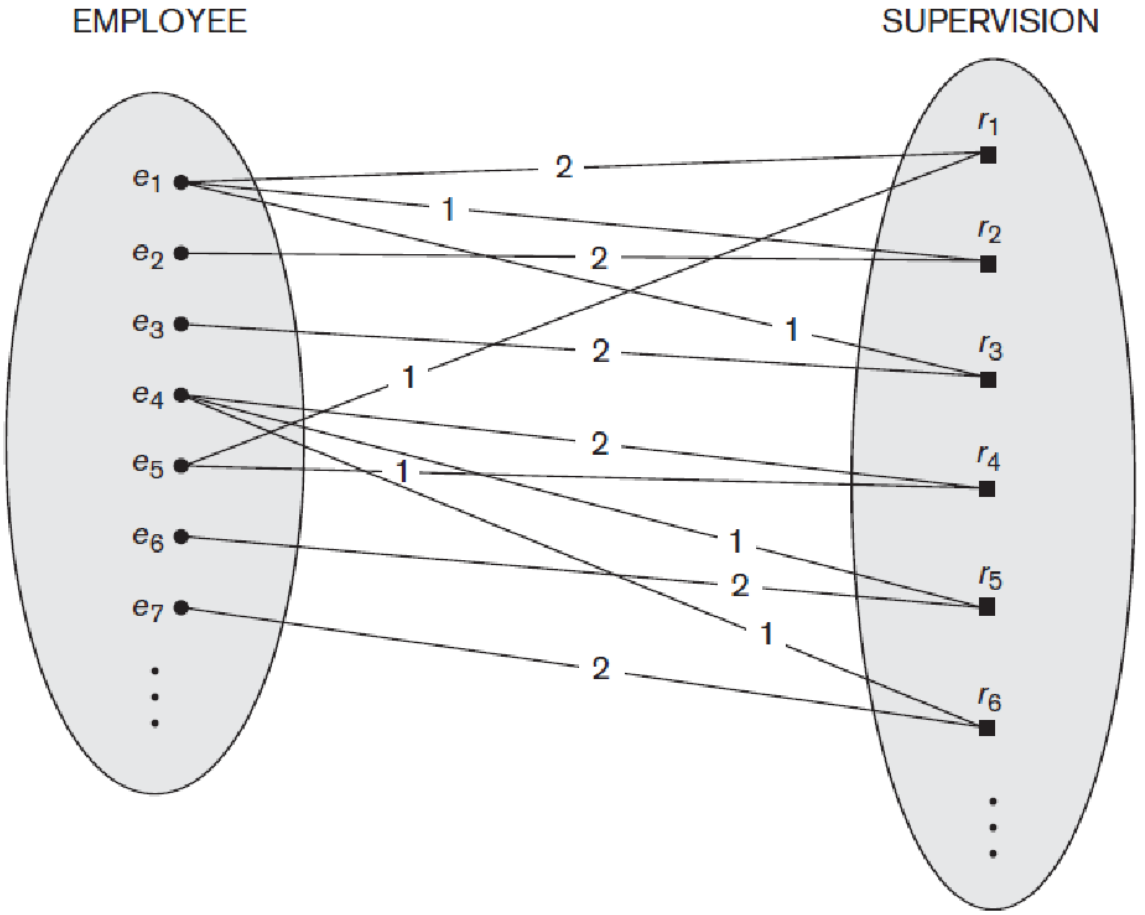


Figure 7.11
A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (10)

Constraints on Relationship Types (1)

- Relationship types usually have **certain constraints** that limit the possible combinations of entities that may participate in the corresponding relationship set.
- These **constraints are determined from the miniworld** situation that the relationships represent.
- For example, if the company has a rule that **each employee must work for exactly one department,** then we would like to describe this constraint in the schema.
- We can distinguish two main types of relationship constraints: **cardinality ratio and participation.**

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (11)

Constraints on Relationship Types (2)

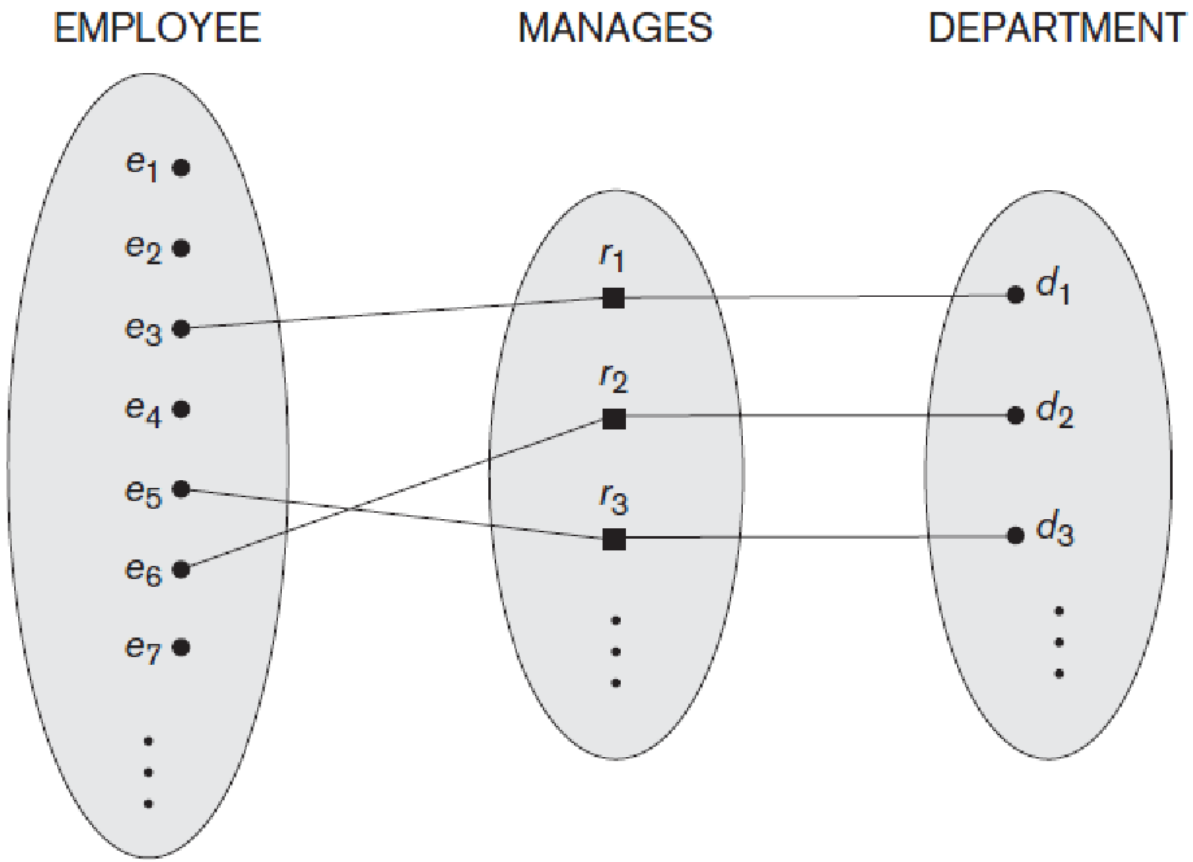
Cardinality Ratios for Binary Relationships.

- The cardinality ratio for a binary relationship specifies the **maximum number of relationship instances that an entity can participate in**.
- For example, in the **WORKS_FOR** binary relationship type, DEPARTMENT: EMPLOYEE is of cardinality ratio l:N, meaning that each department can be related to any number of employees." but an employee can be related to (work for) only one department.
- The possible cardinality ratios for binary relationship types are **1:1, l:N, N:l, and M:N.**
- An example of a **1:1 binary relationship** is **MANAGES**, which relates a department entity to the employee who manages that department.
- The relationship type **WORKS_ON** is of cardinality ratio **M:N**, because the miniworld rule is that an employee can work on several projects and a project can have several employees.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (12)

Constraints on Relationship Types (3)

Figure 7.12
A 1:1 relationship, MANAGES.



RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (13)

Constraints on Relationship Types (4)

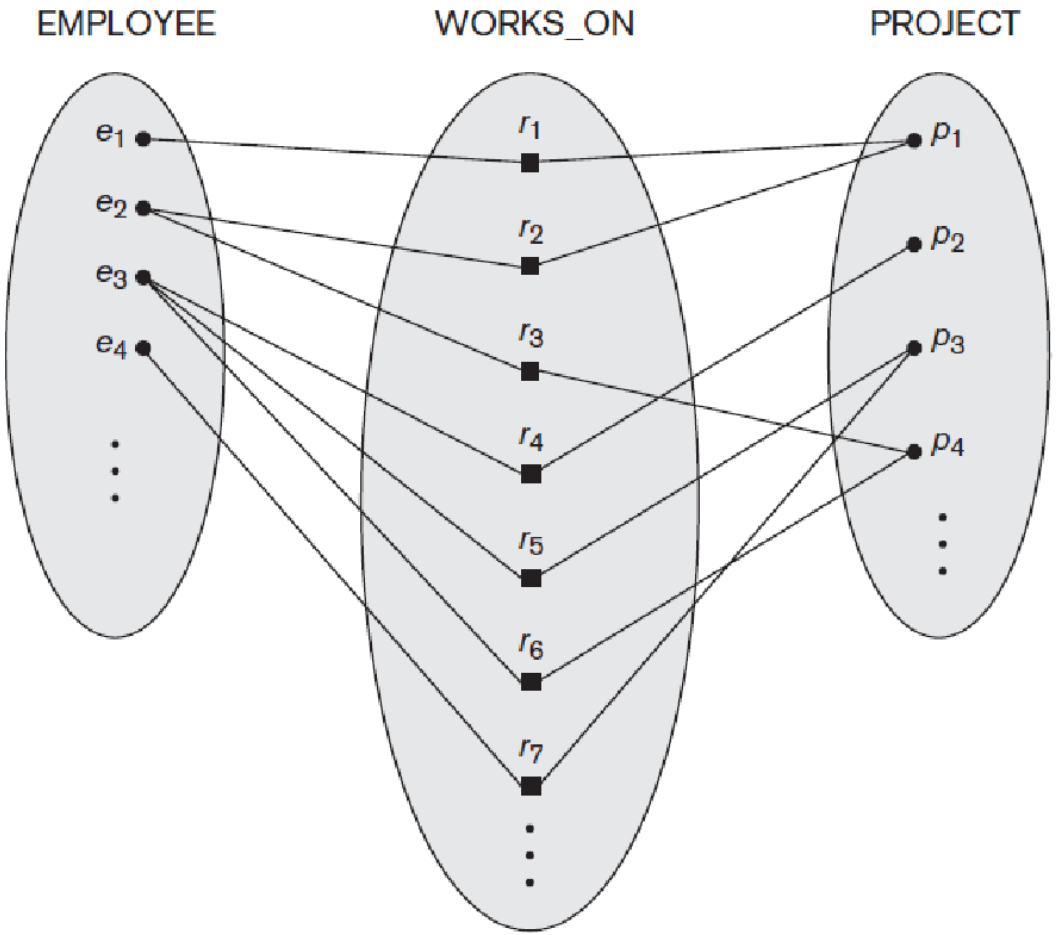


Figure 7.13
An M:N relationship, WORKS_ON.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (14)

Constraints on Relationship Types (5)

- **Participation Constraints and Existence Dependencies.**

- The participation constraint specifies **whether the existence of an entity depends on its being related to another entity via the relationship type.**
- This constraint specifies the minimum number of relationship instances that each entity can participate in, and is sometimes called the **minimum cardinality constraint.**
- There are two types of participation constraints-**total and partial** which we illustrate by example.
- If a company policy states that every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance. Thus, the participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in "the total set" of employee entities must be related to a department entity via WORKS_FOR.
- Total participation is also called **existence dependency.**
- We do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is partial, meaning that some or "part of the set of" employee entities are related to some department entity via MANAGES, but not necessarily all.
- We will refer to the **cardinality ratio and participation constraints,** taken together, as the **structural constraints of a relationship type.**

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (15)

Attributes of Relationship Types (1)

- **Relationship types can also have attributes,** similar to those of entity types.
- For example, to record the number of hours per week that an employee works on a particular project, we can include an **attribute Hours for the WORKS_ON** relationship type.
- Another example is to include the date on which a manager started managing a department via an attribute **StartDate for the MANAGES** relationship type.
- Notice that attributes of 1:1 or I:N relationship types can be migrated to one of the participating entity types.
- For example, the StartDate attribute for the MANAGES relationship can be an attribute of either EMPLOYEE or DEPARTMENT, although conceptually it belongs to MANAGES.
- This is because MANAGES is a 1:1 relationship, so every department or employee entity participates in at most one relationship instance.

RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS (16)

Attributes of Relationship Types (2)

- Hence, the value of the StartDate attribute can be determined separately, either by the participating department entity or by the participating employee (manager) entity.
- For a I:N relationship type, a relationship attribute can be migrated only to the entity type on the N-side of the relationship. For example, if the WORKS_FOR relationship also has an attribute StartDate that indicates when an employee started working for a department, this attribute can be included as an attribute of EMPLOYEE.
- In both 1:1 and I:N relationship types, the decision as to where a relationship attribute should be placed-as a relationship type attribute or as an attribute of a participating entity type-is determined subjectively by the schema designer.
- For M:N relationship types, some attributes may be determined by the combination of participating entities in a relationship instance, not by any single entity. Such attributes must be specified as relationship attributes. An example is the Hours attribute of the M:N relationship WORKS_ON the number of hours an employee works on a project is determined by an employee-project combination and not separately by either entity.

WEAK ENTITY TYPES (1)

- Entity types that **do not have key attributes** of their own are called **weak entity types.**
- In contrast, regular entity types that **do have a key attribute** are called **strong entity types.**
- Entities belonging to a weak entity type are **identified by being related to specific entities from another entity type in combination with one of their attribute values.**
- We call this other entity type the **identifying or owner entity type**, and we call the relationship type that relates a weak entity type to its owner the **identifying relationship** of the weak entity type.
- A **weak entity type always has a total participation constraint** (existence dependency) with respect to its identifying relationship, because a weak entity cannot be identified without an owner entity. However, not every existence dependency results in a weak entity type.

- For example, a DRIVER_LICENSE entity cannot exist unless it is related to a PERSON entity, even though it has its own key (LicenseNumber) and hence is **not a weak entity**.

WEAK ENTITY TYPES (2)

- Consider the entity type DEPENDENT, related to EMPLOYEE, which is used to keep track of the dependents of each employee via a l:N relationship.
- The attributes of DEPENDENT are Name (the first name of the dependent), BirthDate, Sex, and Relationship (to the employee).
- A weak entity type normally has a **partial key**, which is the set of attributes that can **uniquely identify weak entities** that are related to the same owner entity.
- **In the worst case, a composite attribute of all the weak entity's attributes will be the partial key.**
- The partial key attribute is underlined with a dashed or dotted line.
- Weak entity types can sometimes be **represented as complex** (composite, multivalued) attributes.
- In the preceding example, we could specify a multivalued attribute Dependents for EMPLOYEE, which is a composite attribute with component attributes Name, BirthDate, Sex, and Relationship.

WEAK ENTITY TYPES (3)

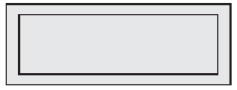
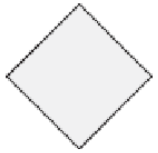
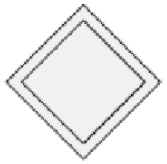
- If the weak entity participates independently in relationship types other than its identifying relationship type, then it should not be modeled as a complex attribute.
- In general, **any number of levels of weak entity types can be defined**; an owner entity type may itself be a weak entity type.
- In addition, a weak entity type may have **more than one identifying entity type and an identifying relationship** type of degree higher than two

ER NOTATION (1)

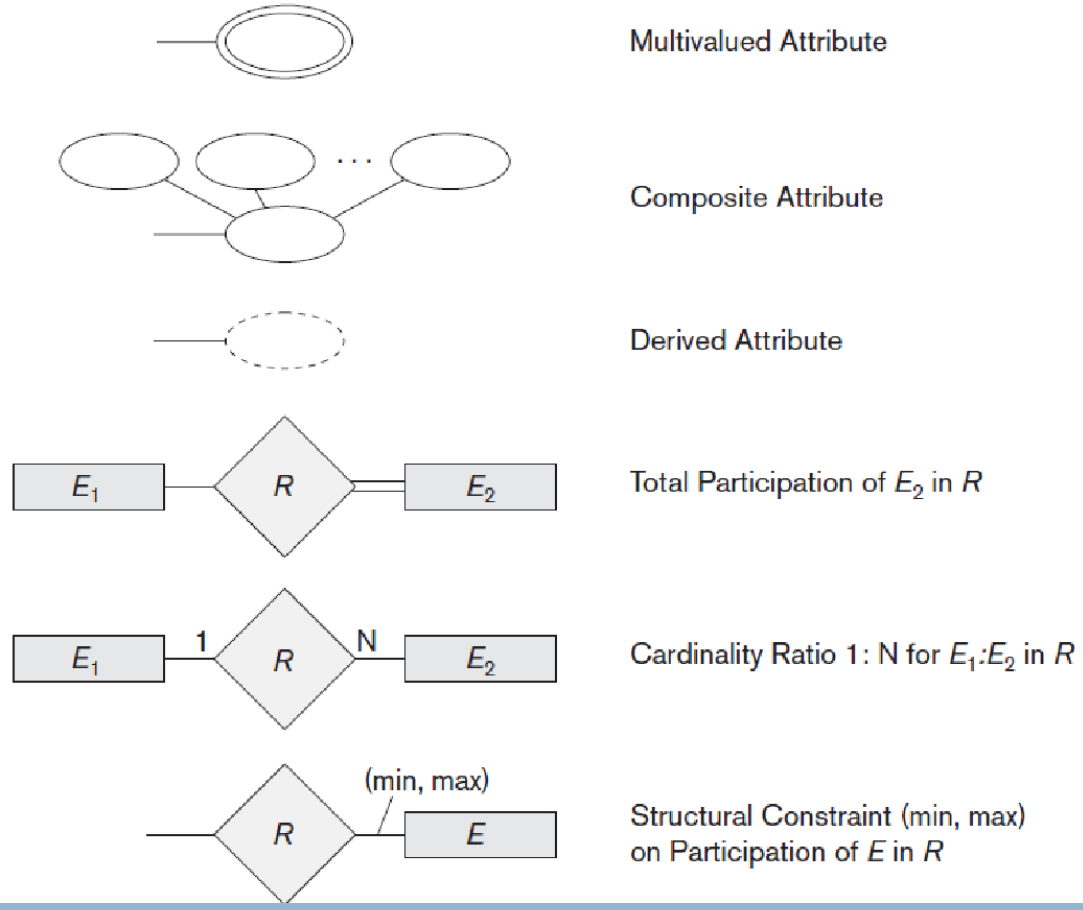
ER Notation

- An entity type is represented in ER diagrams as a rectangular box enclosing the entity type name.
- Attribute names are enclosed in ovals and are attached to their entity type by straight lines.
- Composite attributes are attached to their component attributes by straight lines.
- Multivalued attributes are displayed in double ovals.
- In ER diagrammatic notation, each key attribute has its name underlined inside the oval.
- In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines.

ER NOTATION (2)

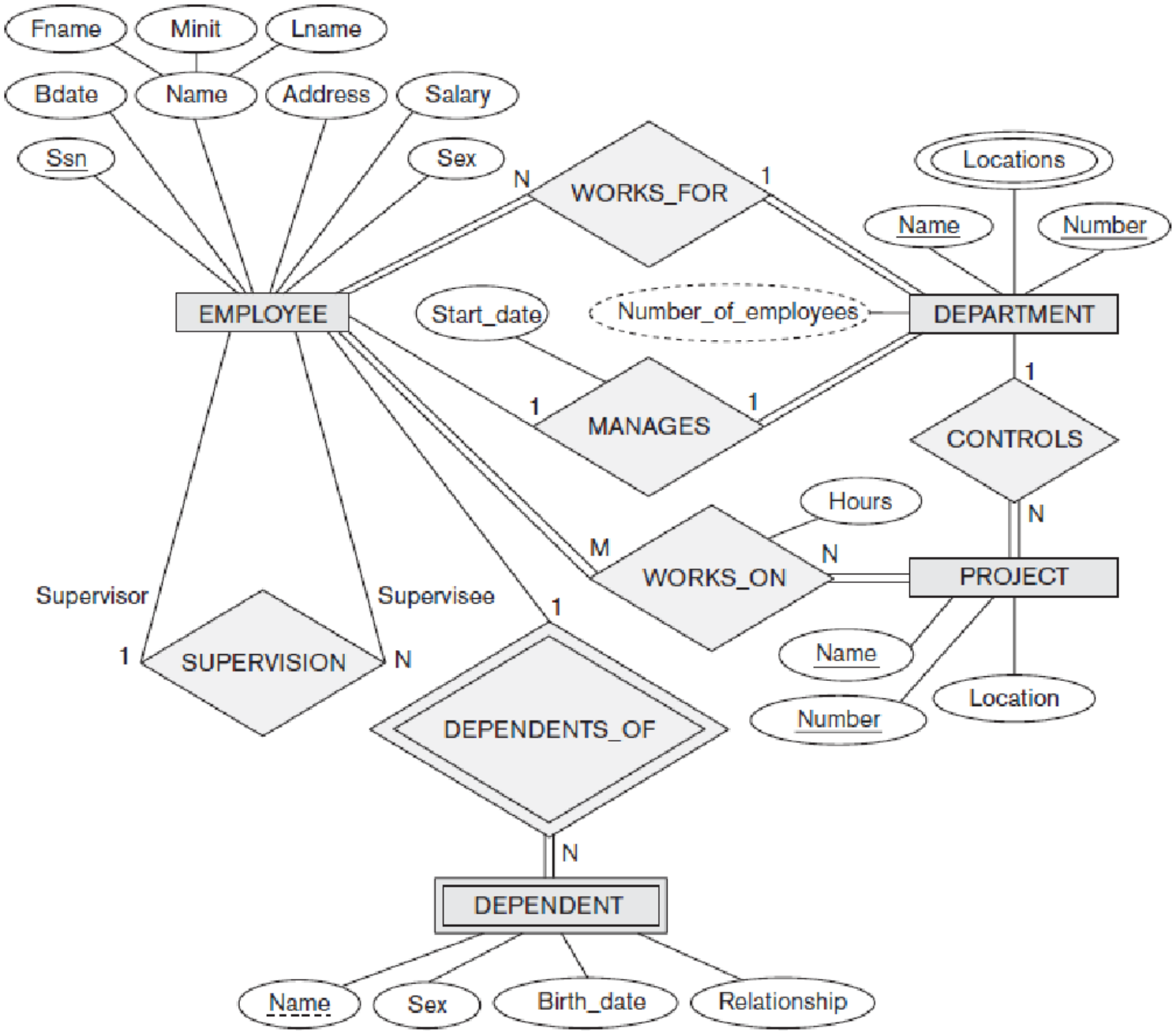
Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Indentifying Relationship
	Attribute
	Key Attribute

ER NOTATION (3)

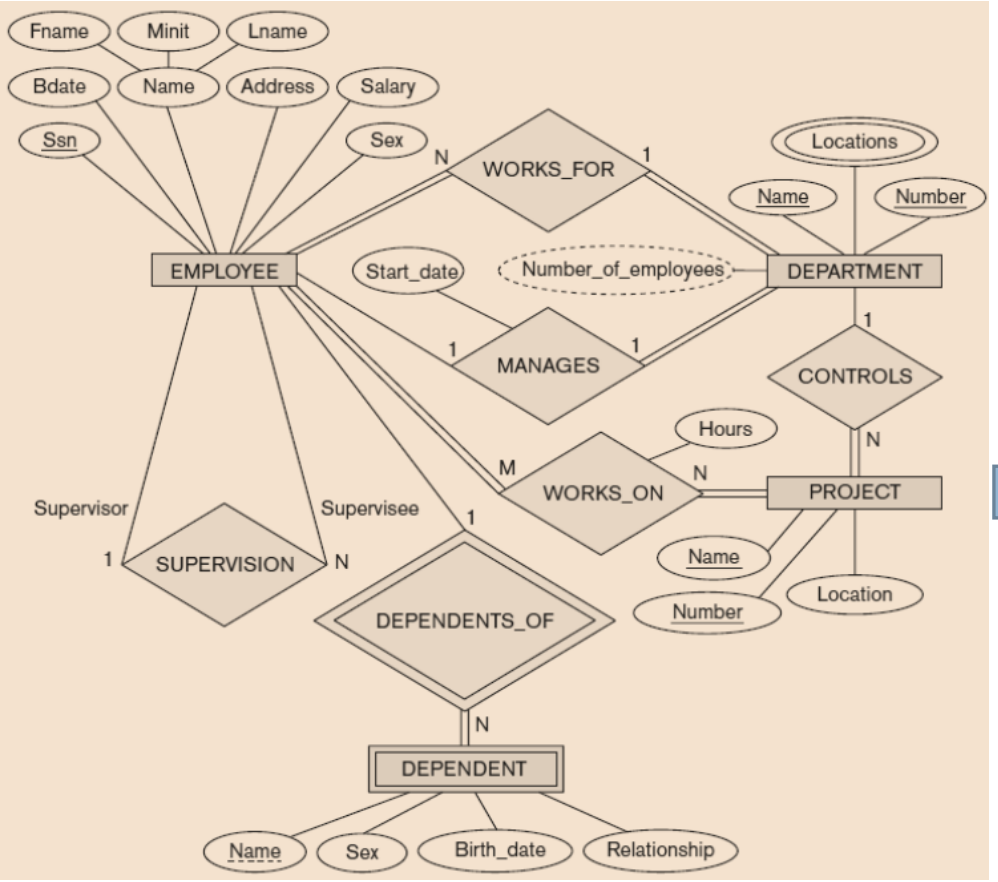


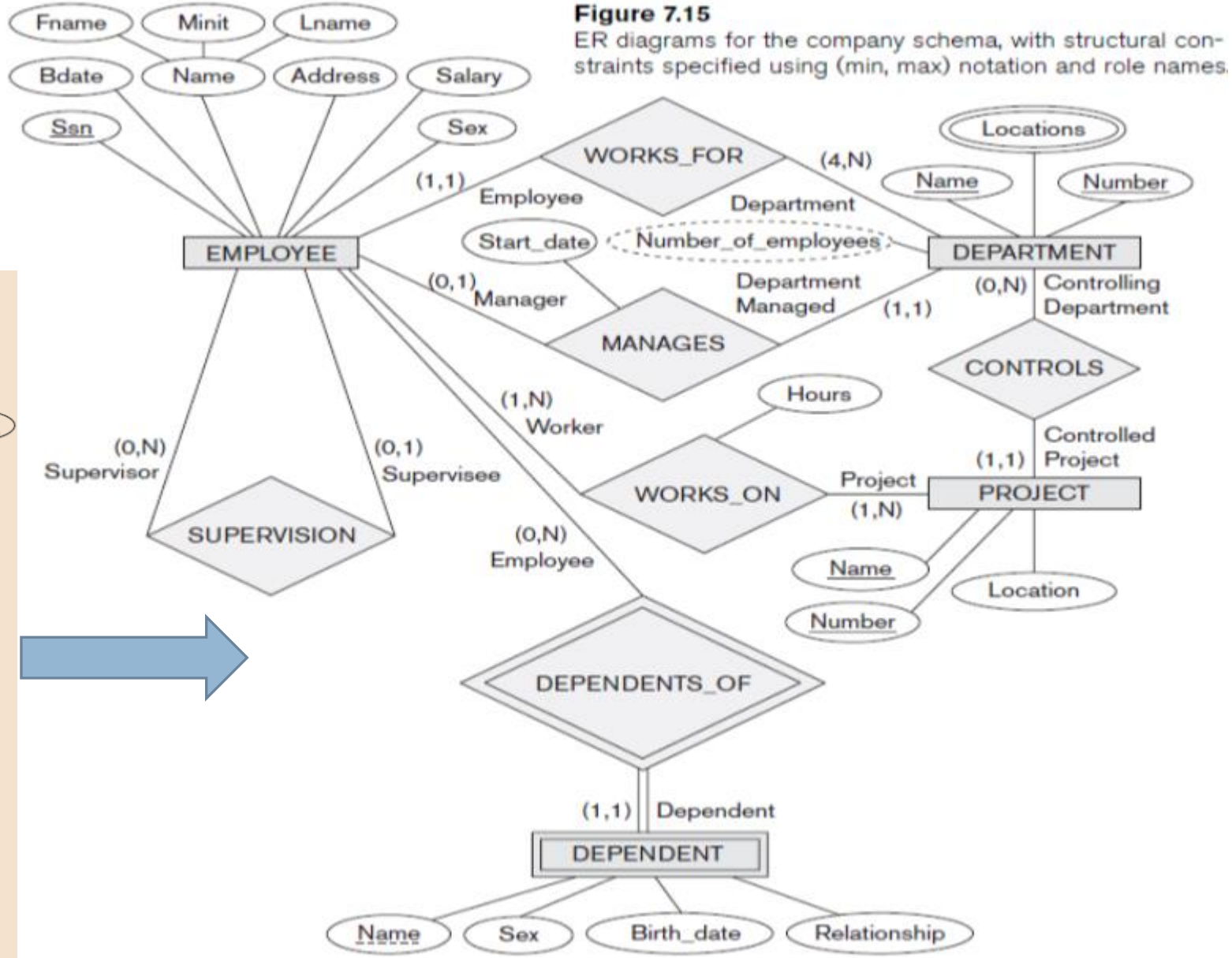
RAMAIAH
Institute of Technology

An ER schema diagram for the COMPANY database.



Min Max Constraints

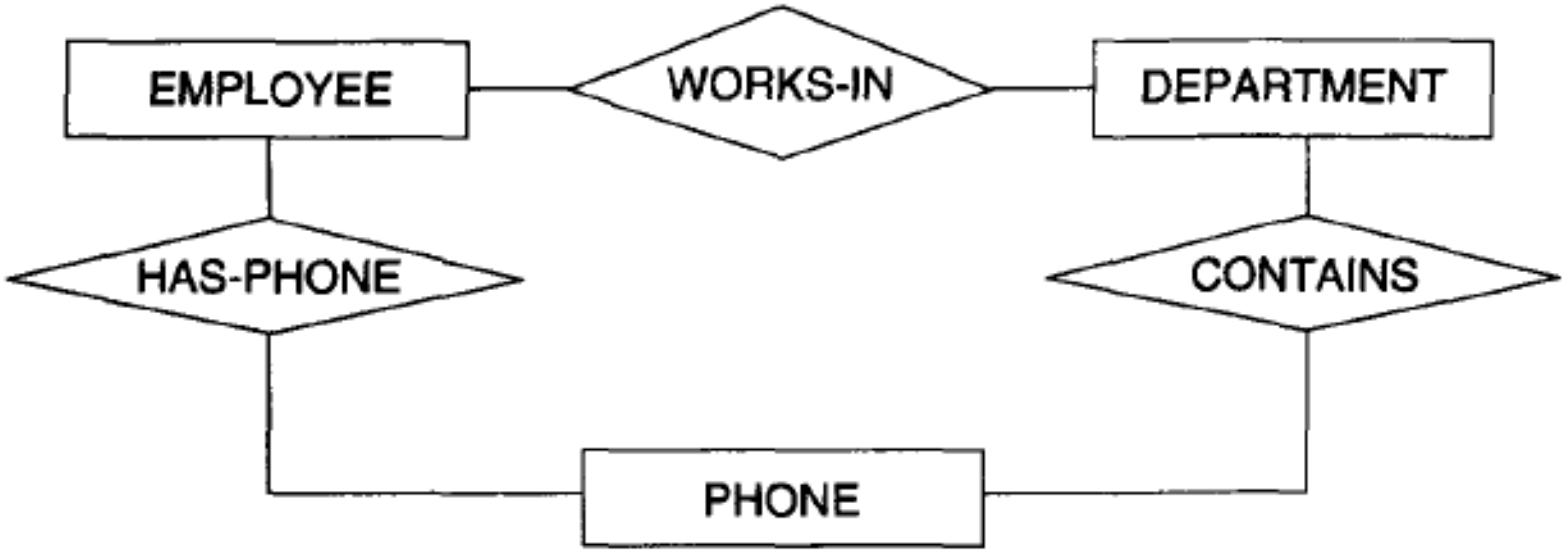




RAMAIAH
Institute of Technology

Tutorial 1

- Consider the ER diagram in Figure . Assume that an employee may work in up to two departments or may not be assigned to any department. Assume that each department must have one and may have up to three phone numbers. Supply (min, max) constraints on this diagram. State clearly any additional assumptions you make. Under what conditions would the relationship HAS_PHONE be redundant in this example?
-



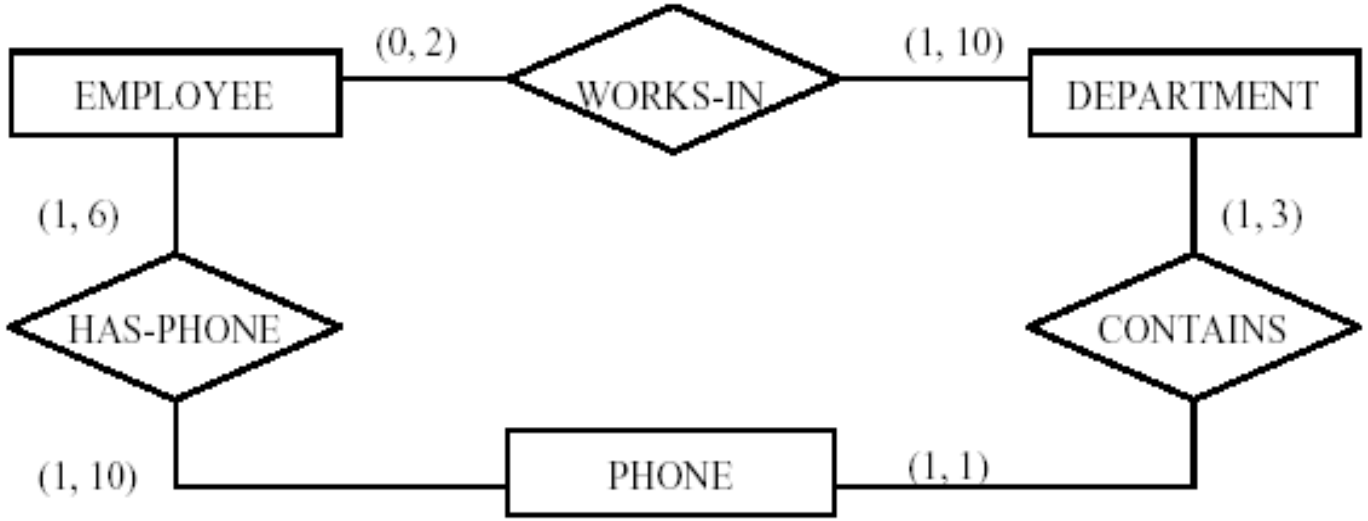
Solution

Assuming the following additional assumptions:

- Each department can have anywhere between 1 and 10 employees.
- Each phone is used by one, and only one, department.
- Each phone is assigned to at least one, and may be assigned to up to 10 employees.
- Each employee is assigned at least one, but no more than 6 phones.

The resulting ER Diagram will have the (min, max) constraints shown in Figure A.

Figure A



RAMAIAH Institute of Technology

Review Questions (1)

3.1. Discuss the role of a high-level data model in the database design process.

3.2. List the various cases where use of a null value would be appropriate.

3.3. Define the following terms: *entity, attribute, attribute value, relationship instance, composite attribute, multivalued attribute, derived attribute, complex attribute, key attribute, value set (domain).*

3.4. What is an entity type? What is an entity set? Explain the differences among an entity, an entity type, and an entity set.

3.5. Explain the difference between an attribute and a value set.

3.6. What is a relationship type? Explain the differences among a relationship instance, a relationship type, and a relationship set.

3.7. What is a participation role? When is it necessary to use role names in the description of relationship types?

RAMAIAH Institute of Technology

Review Questions (1)

3.8. Describe the two alternatives for specifying structural constraints on relationship types. What are the advantages and disadvantages of each?

3.9. Under what conditions can an attribute of a binary relationship type be migrated to become an attribute of one of the participating entity types?

3.10. When we think of relationships as attributes, what are the value sets of these attributes? What class of data models is based on this concept?

3.11. What is meant by a recursive relationship type? Give some examples of recursive relationship types.

3.12. When is the concept of a weak entity used in data modeling? Define the terms *owner entity type*, *weak entity type*, *identifying relationship type*, and *partial key*.

3.13. Can an identifying relationship of a weak entity type be of a degree greater than two? Give examples to illustrate your answer.

3.14. Discuss the conventions for displaying an ER schema as an ERdiagram.

3.15. Discuss the naming conventions used for ER schema diagrams.

End of chapter 3